

TP1 : Utilisation du logiciel R*

Qu'est-ce que R ?

R est un logiciel permettant de faire des analyses statistiques et de produire des graphiques. Il est adapté pour la manipulation et l'analyse des données, les calculs matriciels, l'utilisation de méthodes statistiques classiques et avancées, et les représentations graphiques. C'est également un langage de programmation performant et simple à utiliser. Il dispose de fonctions prédéfinies et permet d'écrire ses propres fonctions. R est un logiciel libre ce qui signifie que les utilisateurs ont la liberté d'exécuter, de copier, de distribuer, d'étudier et améliorer ce logiciel. R est disponible pour les systèmes d'exploitation Unix, Linux, Windows et MacOSX sur le site CRAN (<http://www.r-project.org/>). De nombreux tutoriels existent sur le web, par exemple *R pour les débutants* de E. Paradis.

RStudio

R fonctionne en ligne de commande, mais des interfaces permettent une utilisation plus conviviale, comme RStudio (<https://www.rstudio.com/>). L'interface RStudio est composée de 4 fenêtres :

1. Fenêtre de commande ou console (à gauche) : contient une console dans laquelle les codes R sont saisis et exécutés.
2. Fenêtre espace de travail (en haut à droite) : contient les objets en mémoire, que l'on peut consulter en cliquant sur leur nom, et l'historique des commandes exécutées.
3. Fenêtre explorateur (en bas à droite) : permet de se déplacer dans les répertoires, de visualiser les graphiques, montre les packages installés et permet d'installer de nouveaux packages, permet d'accéder à la documentation.
4. Fenêtre édition (en haut à gauche) : cette fenêtre n'apparaît que s'il existe des fichiers contenant les scripts R ouverts. Il est recommandé de taper son code dans cette fenêtre et de l'exécuter ensuite dans la console afin de pouvoir sauvegarder le code.

R Markdown

Pour la rédaction d'un projet par exemple, l'utilisation de R Markdown dans RStudio est pratique. Cela vous permettra de mettre des commentaires entre les lignes de commandes.

1. Dans RStudio cliquez sur File → New File → R Markdown.
2. Choisissez pdf comme Default Output Format.
3. Un nouveau fichier Markdown s'ouvre avec un exemple qui permet de comprendre la syntaxe.
4. Pour produire le pdf, il faut cliquer sur Knit PDF.

Vous pouvez aller voir le lien suivant pour plus de détails : (<http://larmarange.github.io/analyse-R/rmarkdown-les-rapports-automatisees.html>).

*inspirée de Christine Keribin

Les packages

R est organisé en packages: les packages de base sont chargés en mémoire au moment du lancement, certains seront à charger avant utilisation par la commande `library()`. D'autres peuvent ne pas être encore installés; leur installation pourra se faire grâce à l'onglet Package/Install si l'ordinateur possède une connexion internet. La documentation et l'aide à l'utilisation d'une fonction ou d'un package sont accessibles en cliquant sur l'onglet help ou par ligne de commandes:

```
help("mean")
```

Que peut-on dire des résultats des commandes suivantes :

```
?mean  
??mean  
apropos("mean")
```

Cette séance est une initiation au langage R. A chaque nouvelle séance, **les commandes déjà vues dans les TP précédents seront supposées connues.**

Enfin, le but de cet enseignement n'est pas de devenir un développeur en R mais de savoir écrire et lire un code R pour illustrer et comprendre des concepts et méthodes statistiques.

1. Notions de base

1.1 Assigner une valeur

Pour assigner une valeur à une variable :

```
x<-20  
x=20 #autre commande
```

Vous pouvez ensuite afficher la valeur d'une variable :

```
x  
print(x) #autre commande
```

Calculer le carré du nombre d'or $\left(\frac{1+\sqrt{5}}{2}\right)^2$, stocker sa valeur dans un objet R et afficher sa valeur.

Vous pouvez afficher l'ensemble des variables du répertoire courant contenant des informations :

```
objects()  
ls() # autre commande
```

Les informations précédentes sont disponibles dans la fenêtre **Environment**. Lorsque vous n'avez plus besoin d'une variable, vous pouvez l'effacer. C'est important si elle contient beaucoup d'information et risque de surcharger la mémoire :

```
rm(x) # suppression de l'objet x
```

Il est conseillé de nettoyer l'environnement de travail au début de chaque nouvelle étude:

```
rm(list=objects()) # suppression de tous les objets en mémoire
```

1.2 Répertoire de travail

Le répertoire de travail est par défaut celui à partir duquel l'interface RStudio est lancée. Il est pratique de se placer dans un répertoire de travail bien défini, celui qui contiendra les fichiers .R sur lesquels vous

travaillez. Pour ce faire, vous pouvez soit utiliser la commande `setwd()`, soit utiliser le menu de l'interface.

```
getwd() # indique le répertoire de travail en cours
setwd("MonRepertoire") # définit le répertoire en cours
```

1.3 Fichier de script

Toutes les commandes peuvent être lancées à partir d'un fichier.

1. Créer un fichier de commandes monTP.
2. L'enregistrer avec l'extension .R.
3. Y copier les quelques commandes précédemment exécutées depuis l'onglet **History** de la fenêtre **Environment** (icône **To Source**).
4. Enregistrer le fichier.
5. Pour exécuter le code depuis la fenêtre d'édition, sélectionner les commandes à exécuter, puis cliquer sur l'icône run, ou utiliser le raccourci clavier CTL+enter (command+enter sur Mac). Note: ce raccourci clavier permet également d'exécuter la ligne sur laquelle se trouve le curseur, sans avoir à la sélectionner en entier.
6. Il est possible d'exécuter la totalité fichier (même non ouvert) avec la commande `source('monTP')`.
7. Le fichier TP1.R sous Edunao contient les commandes présentes sur la feuille de TP. Vous pouvez le télécharger dans votre répertoire de TP, puis l'éditer dans R pour vous faire gagner du temps.

2. Quelques objets R

Les objets de R se différencient par leur mode, qui décrit leur contenu (**logical**, **numeric**, **character**), et leur classe, qui décrit leur structure (**vector**, **matrix**, **array**, **factor**, **data.frame**, **list**, **function**,...). Les listes ou les data.frame peuvent avoir un contenu hétérogène par sous-structure. Les nombres décimaux sont encodés avec un point décimal (**3.14**), les chaînes de caractères par des guillemets simples ou doubles (**'BLEU'**), et les valeurs logiques par **TRUE** ou **FALSE**, ou leurs abréviations **T** et **F**. Les valeurs manquantes sont codées par la chaîne de caractères **NA**.

2.1 Vecteurs

L'objet **x** défini ci-dessus est un scalaire. Pour définir un vecteur, la fonction de base est la fonction de concaténation `c()`, mais il existe des fonctions pour créer des suites régulières: `seq()`, `rep()` ou l'opérateur `:` par exemple.

Expliquer le résultat des commandes suivantes:

```
c(3,9,5)
c(1,c(2,3,4),5)
2:12
5:-3
x = c(4,5,6); y = c(2,-2,5)
x + y; -x; x > 5
min(x); sin(x); sort(x); length(x)
z = c("I", "love", "maths")
```

Une partie des éléments d'un vecteur peut être sélectionnée en ajoutant après le vecteur un vecteur indice entre crochets :

```
x = c(5,6,7,2,3,4)
x[c(1,3,5)] # Indices positifs
x[-3] # Indice négatif, exclut le 3ème élément de x
x[c(T,T,F,F,T,T)] # Indice valeurs logiques
x[x > 5]
```

1. Construire les vecteurs suivants avec la méthode la plus adaptée
 - $V_1 = (-1, 3.2, -2, 8)$
 - $V_2 = (-2, -1, 0, 1, 2, 3, 4, 5, 6)$
 - $V_3 = (0.05, 0.1, 0.15, 0.2)$
 - $V_4 = (1, 1, 1, 1, 1, 1, 1, 1, 1)$
 - $V_5 = (\text{OUI}, \text{NON})$
2. Ordonner le vecteur V_1 .
3. Quel est le résultat de l'opération $V_6 = 2 * V_2 - 3$, puis de l'opération $V_3 + V_2$? Qu'en concluez-vous sur les opérations arithmétiques utilisant des vecteurs ? Calculer le logarithme des coordonnées de V_3 . Quel est le résultat de $V_5 + 1$?
4. Accéder à la deuxième composante de V_5 . Afficher la longueur de V_6 . Créer le vecteur V_7 formé des 3 dernières composantes de V_6 .
5. Calculer la somme des coefficients de V_6 .
6. Application: la table 1 donne, pour l'année 1982, le pourcentage des communes qui disposent d'une boulangerie en fonction de la taille de la commune. (Source : INSEE-SCEES, inventaire communal 1988).
 - Créer un vecteur nbc pour enregistrer les valeurs de la colonne 2 et un vecteur pctb pour enregistrer les valeurs de la colonne 3.
 - Calculer le pourcentage de communes qui disposent d'une boulangerie sur l'ensemble du territoire.

Nombre d'habitants	Nombre de communes	Boulangerie (%)
moins de 100	4138	1.1
100 à 199	7077	6.6
200 à 499	11176	26.3
500 à 999	6474	64.7
1000 à 1999	3735	88.7
2000 à 4999	2365	98.0
plus de 5000	1573	99.9

Table 1: Pourcentage de communes qui disposent d'une boulangerie

2.2 Structure de données

La meilleure façon de décrire et analyser des données dans R est de les saisir dans des objets appelés **data.frame**. Un data.frame possède des lignes et des colonnes comme une matrice, mais les colonnes peuvent être de nature différente : numérique, caractères, logique. Les lignes contiennent différentes observations ou mesures et peuvent être vues comme des "individus" ou "unités statistiques". Les colonnes contiennent les valeurs de variables afférentes aux individus. Elles ont des noms, ce qui permet d'y accéder par l'opérateur `$`.

Expliquer le résultat des commandes suivantes:

```
df= data.frame(n=nbc,pctb,
taille=c("<100","100 à 199","200 à 499","500 à 999","1000 à 1999","2000 à 4999",
"plus de 5000"))
```

```

print(df)
dim(df)
nrow(df)
names(df)

head(df)
str(df)
summary(df)

df$pctb
df["pctb"] # différence avec le précédent ?
max(df$pctb)
df[1:4,2]

attach(df) # accès direct aux colonnes, mais attention !!
detach(df)

```

Nous étudions maintenant le jeu de données **genotype** du package **MASS**, qu'il faut commencer par charger.

1. Afficher le résumé de chaque variable.
2. Modifier l'instruction pour sélectionner les portées dont le génotype de la mère est A et celui de la portée est B.
3. Quelle est la fonction de la commande **tapply()** ? Calculer le poids moyen suivant le génotype de la mère et de la portée, puis le poids moyen général.

```

library(MASS)
?genotype
genotype[ genotype$Litter=="I" | genotype$Mother=="A" ,]
?tapply
tapply(genotype$Wt,genotype$Mother,mean)

```

3. Graphiques

Le logiciel R propose de nombreuses fonctionnalités graphiques, dont un aperçu peut-être visualisé en jouant la commande **demo(graphics)**. Nous manipulons dans ce TP les fonctions “historiques” de R pour concevoir des graphiques.

Il y a deux types de fonctions graphiques: les fonctions principales (**plot**, **boxplot**, **hist**, etc...) qui définissent le type de graphique, ses axes, etc, et les fonctions secondaires (**points**, **lines**, **text**, **legend**,...) qui ajoutent des ordres graphiques sur un graphe existant.

1. Interpréter le code suivant, indiquer le rôle de chaque argument :

```

plot(genotype$Litter)
Litter.table=table(genotype$Litter)
par(mfrow=c(1,2),oma=c(0,0,3,0))
barplot(Litter.table,main="diagramme en barres", col=rainbow(4))
pie(Litter.table,col=rainbow(4),main="camembert")
title(main="Répartition des génotypes des portées", outer=TRUE)

```

2. Modifier le code pour tracer des barres horizontales.
3. Modifier le code pour que les barres représentent des proportions.

```
par(mfrow=c(1,1))
plot(Wt)
hist(Wt)
paste("moy=", round(mean(Wt),2))
```

4. Interpréter les graphiques générés par le code précédent.
5. Afficher l'histogramme avec l'échelle des ordonnées définie en proportion, une couleur grisée et un titre indiquant l'espérance et l'écart-type de ces données. Ajouter un titre qui affiche la moyenne et l'écart-type.
6. Afficher la boîte à moustaches (**boxplot()**) de la variable Wt. Superposer un point (**points()**) représentant la moyenne de l'échantillon. Commenter.

4. Import et export de données

4.1 A partir d'un fichier texte

L'import et l'export de fichiers de données sous forme de fichier texte peuvent se faire en appelant les fonction **read.table()** (import dans un data.frame d'un fichier texte) et **write.table()** (export dans un data.frame d'un fichier texte). Par exemple, la commande suivante enregistre le data.frame des boulangeries dans un fichier texte, en omettant les guillemets pour les chaînes de caractères et en choisissant le point virgule comme séparateur :

```
write.table(df,"boulangeries.txt",row.names=FALSE,sep=";",quote=FALSE)
```

Si le fichier contient des lignes que l'opération de lecture doit ignorer (par exemple, une explication du contenu du fichier), il faut indiquer de les sauter (**skip=**). L'argument **header=** indique si la ligne juste avant les données contient le nom des variables.

Modifier le fichier **boulangeries.txt** en ajoutant une ligne de commentaire indiquant le contenu du fichier, puis importer le fichier texte dans un nouveau data.frame. Vérifier le résultat obtenu.

```
df2=read.table("boulangeries.txt",sep=";",header=TRUE,skip=1)
```

4.2 A partir d'un fichier binaire

Il est également possible d'exporter **save()** et d'importer **load()** des objets suivant un format binaire propre à R. Dans ce cas, il est commode de définir le fichier enregistré avec une extension **.RData** pour permettre une identification aisée du type de fichier.

Le code suivant enregistre le data.frame sous forme binaire, le supprime de l'environnement R puis le recharge à partir de son enregistrement :

```
save(df,file="boulangeries.RData")
rm(df)
load("boulangeries.RData")
```

En particulier, à l'issue de la session, RStudio propose d'enregistrer tous les objets en mémoire, qui seront alors rechargés au lancement de la session suivante. Il est conseillé de ne pas le faire, et d'enregistrer ce choix dans les options.

Paramétrer le panel des options (menu tools/options) comme suit:

Save workspace to .RData on exit : Never.

5. Une analyse statistique

Le fichier **mineral.data** enregistre les taux de nitrates de dix bouteilles d'eau minérale en mg/ℓ.

1. Lire les données, calculer les statistiques pertinentes. Les représenter graphiquement sous forme de boîte à moustache et y superposer la valeur moyenne. Commenter.
2. Représenter l'histogramme, superposer (**curve**) la densité gaussienne (**dnorm**) de paramètres l'espérance et la variance empirique. Peut-on considérer les données comme étant gaussiennes (**qqnorm**, **shapiro.test**, **ks.test**)?
3. Calculer un intervalle de confiance de l'espérance de niveau 90%, puis 95%, d'abord manuellement, puis avec la fonction **t.test**.
4. Le fournisseur affirme que son eau contient moins de 3.53mg de nitrate par litre. Le constructeur sous-estime-t-il la teneur en nitrates?

6. Matrices

Les matrices ont deux dimensions, sont de mode quelconque mais homogène (ne contiennent que des éléments de même nature). La création d'une matrice se fait avec la syntaxe :

```
n=4 # nombre de lignes
p=3 # nombre de colonnes
matrix(vec,nrow=n,ncol=p)
```

où **vec** est le vecteur contenant les éléments de la matrice, qui seront rangés en colonne (sauf si l'argument **byrow=TRUE** est choisi). On peut également créer des matrices en concaténant des vecteurs de même dimension en ligne (**rbind()**) ou en colonne (**cbind()**).

1. Créer les matrices A et B par concaténation de plusieurs vecteurs (**cbind()**, **rbind()**):

$$A = \begin{pmatrix} 4 & -1 & 1 \\ 3 & 10 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 4 \\ 3 & 3 \\ 4 & 2 \end{pmatrix}.$$

2. Sélection de la deuxième colonne de A, et le coefficient (3,2) de B ?
3. Quel est le résultat des instructions $A * B$, $A[, 1 : 2] * B[2 : 3,]$ et $M = A \%*\% B$?